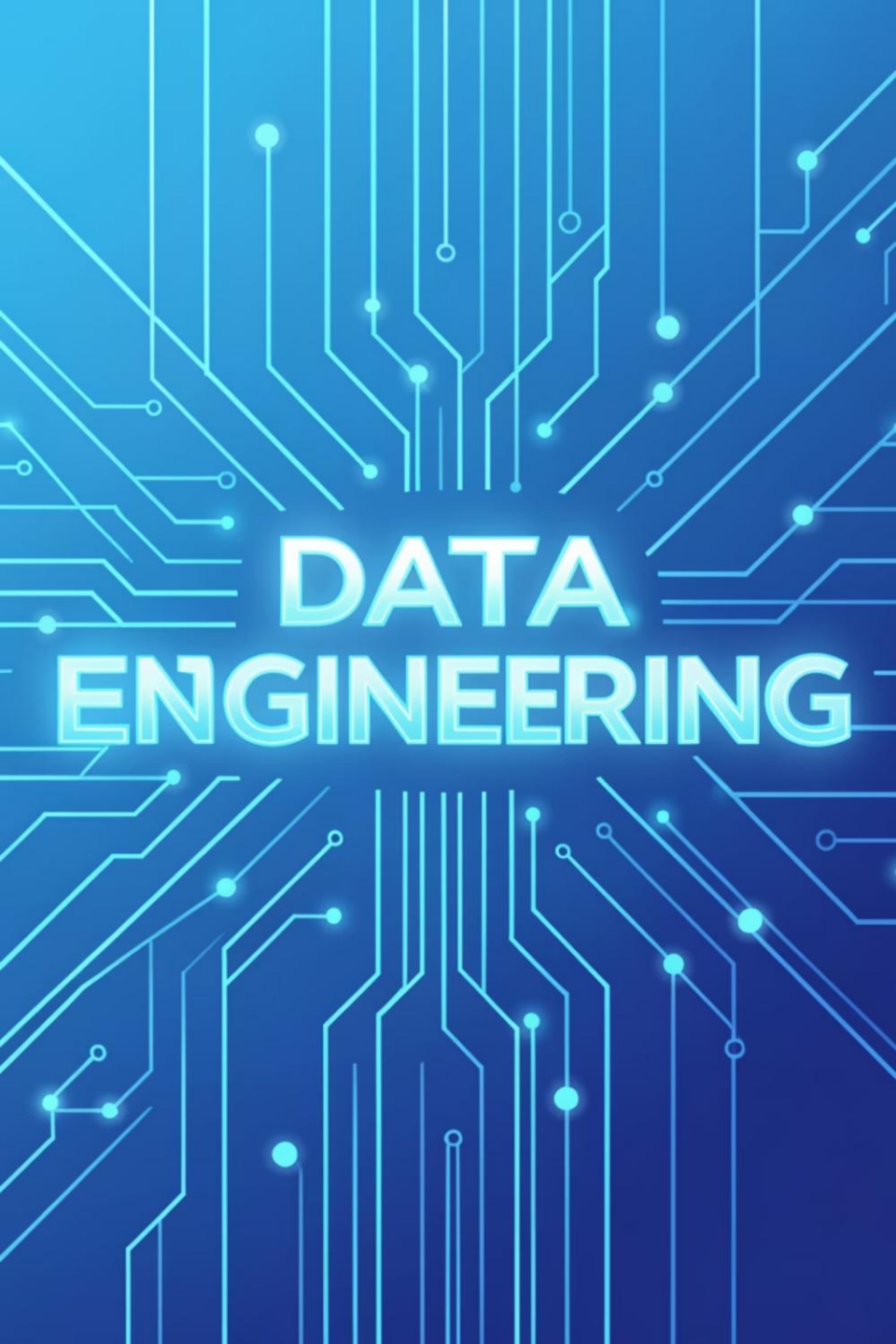




DATA ENGINEERING

Data Engineering Spring '26 – DS463

Feb 26, 2026

Lecture 18

Logistics: Your Course Essentials

Class Timings

Ì ÖŎŇVŘĚ Ć ĎĚĬ PŇCĚŇVŘĚ Ć ĎĚĬ
ĈĈĚĆ Ė Ĭ a ĈĈĚĆ Ĭ Ĭ

Friday (LH8): 11:00 AM – 11:50 AM

Office Hours

Monday, Tuesday: 10:00 AM – 11:00 AM,
AM, S18 2nd Floor NAB or by
appointment.

Credit Hours

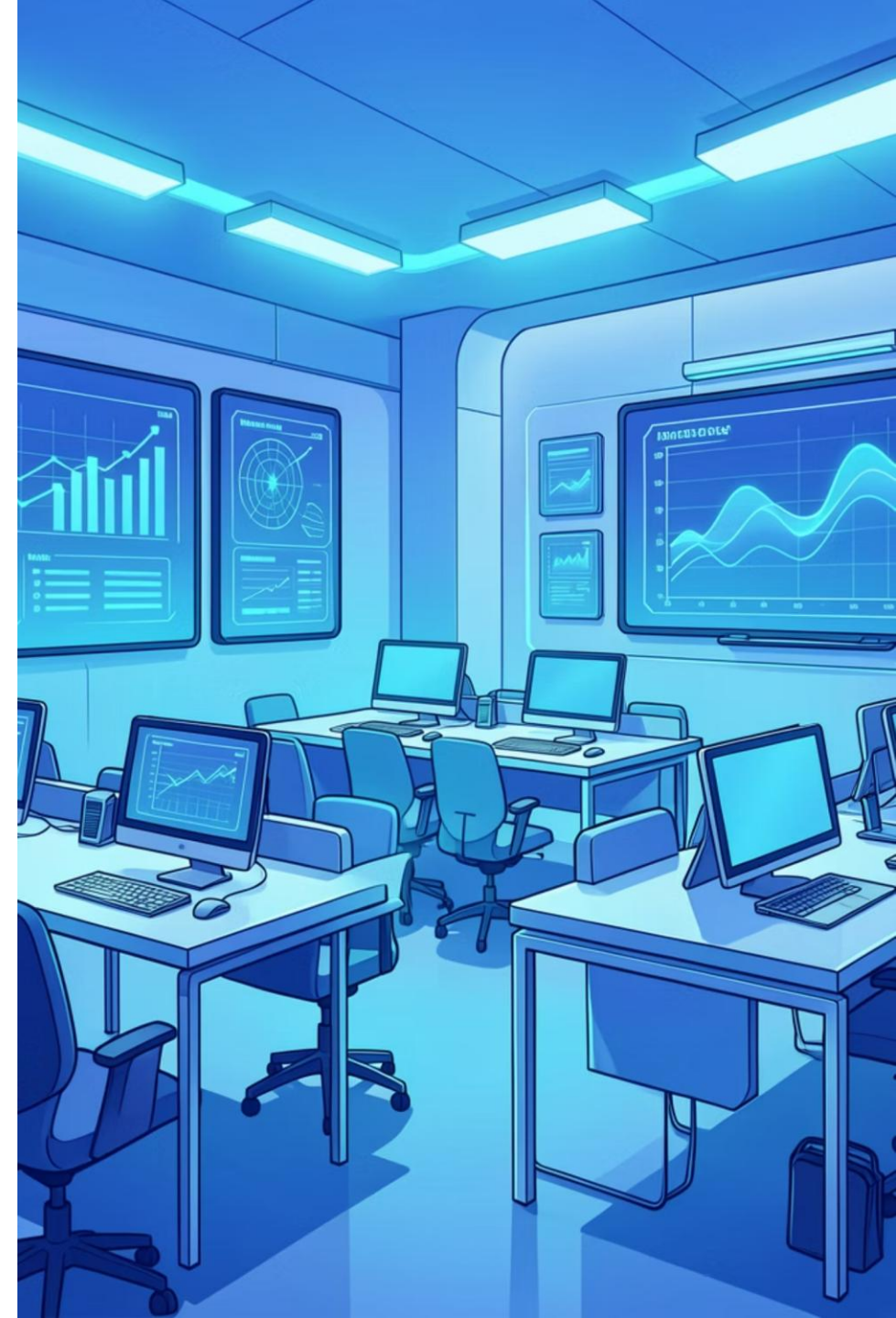
2 hours (Lectures) + 1 hour (Practical
(Practical Session)

Contact & Support

MS Teams: [w5t93yk](#)

[GÖMÖŇVŘĚ Ć ĎĚĬ PŇCĚŇVŘĚ Ć ĎĚĬ](#)

WhatsApp: Reach out for quick queries



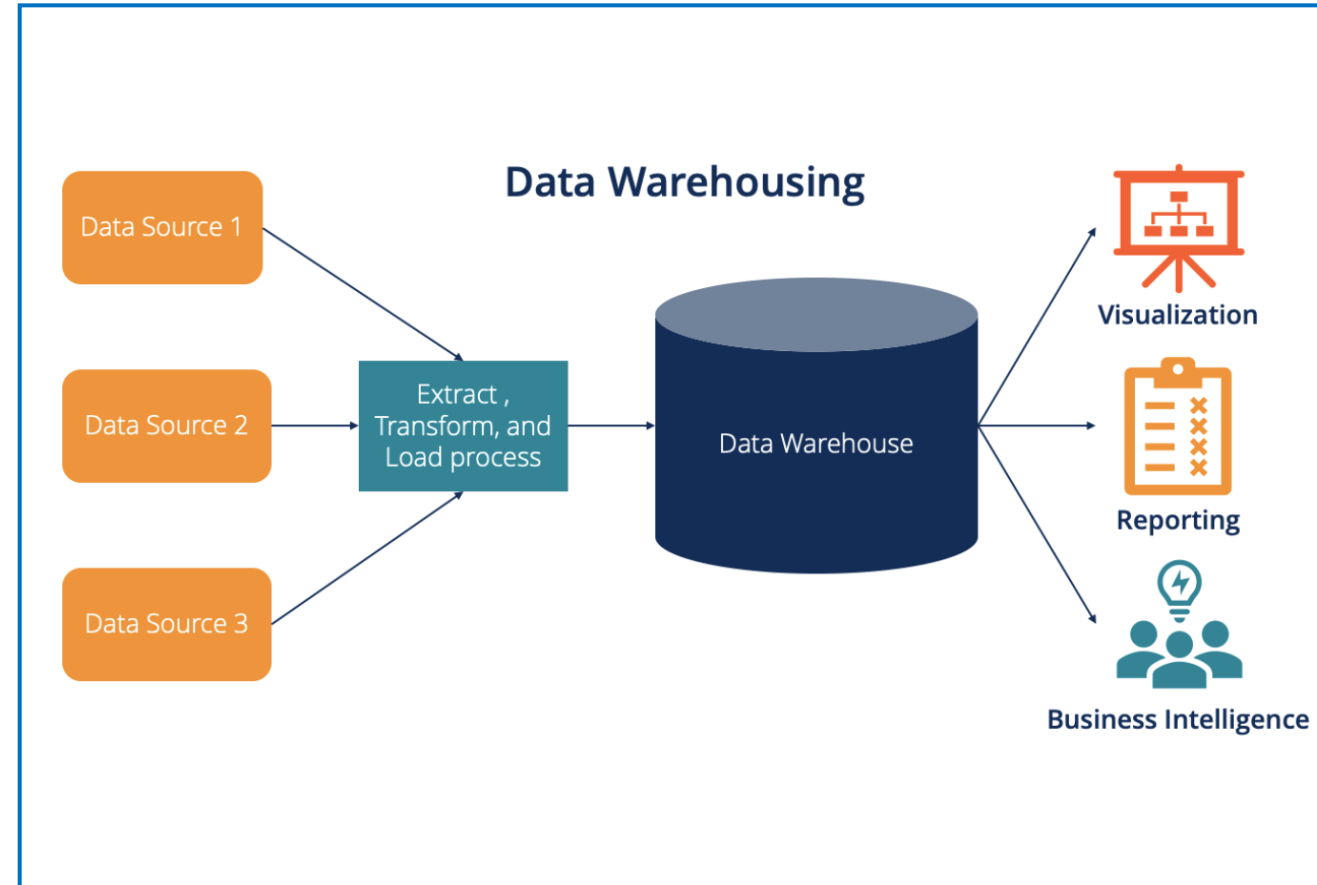
Major Architecture Concepts

- **Domain and Services**
- **Distributed Systems, Scalability and Design for failure**
- **Tight vs Loose Coupling**
- **User Access**
- **Event Driven Architecture**
- **Brownfield vs Greenfield Projects**

Examples and Types of Data Architecture

Data warehouse

- A central data hub used for reporting and analytics.
- Data is highly structured and formatted for analysis.
- First defined in 1989 by Bill Inmon as:
 - Subject-oriented
 - Integrated
 - Nonvolatile (not frequently changed)
 - Time-variant (tracks historical data)

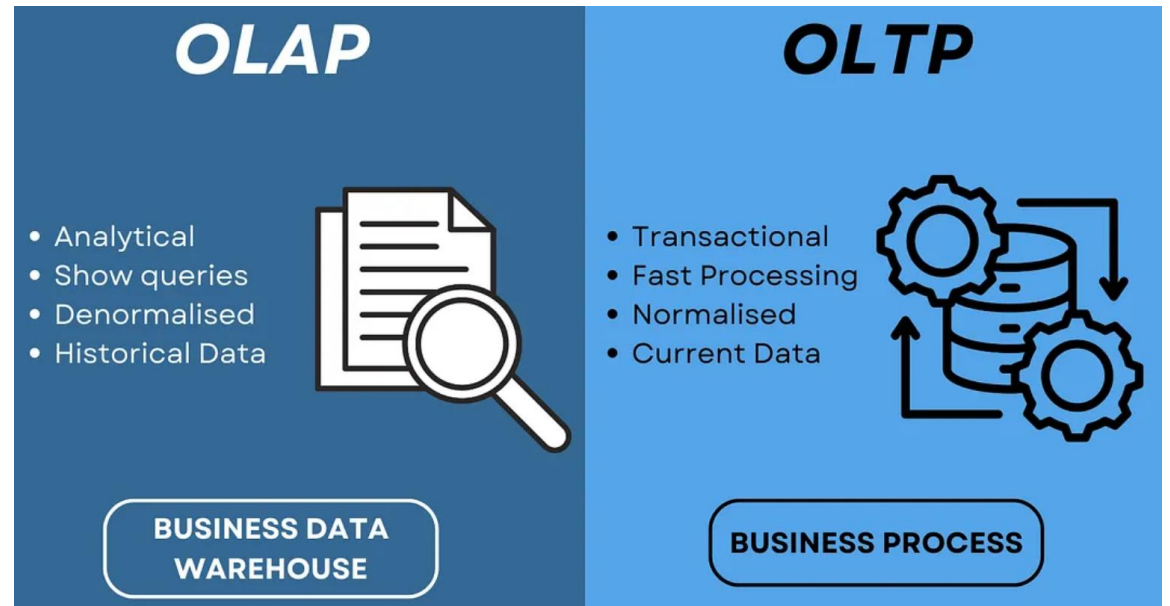


Examples and Types of Data Architecture

Data warehouse

Characteristics

- Separates OLAP from OLTP
- OLTP = transactional systems (production databases).
- OLAP = analytics systems
- Separating them:
 - Protects production systems.
 - Improves analytics performance.



Examples and Types of Data Architecture

Data warehouse

MPP (Massively Parallel Processing)

Distributes queries across many machines.

Optimized for:

- Large data scans
- Aggregations
- Statistical calculations

Uses SQL (like relational databases).

Examples and Types of Data Architecture

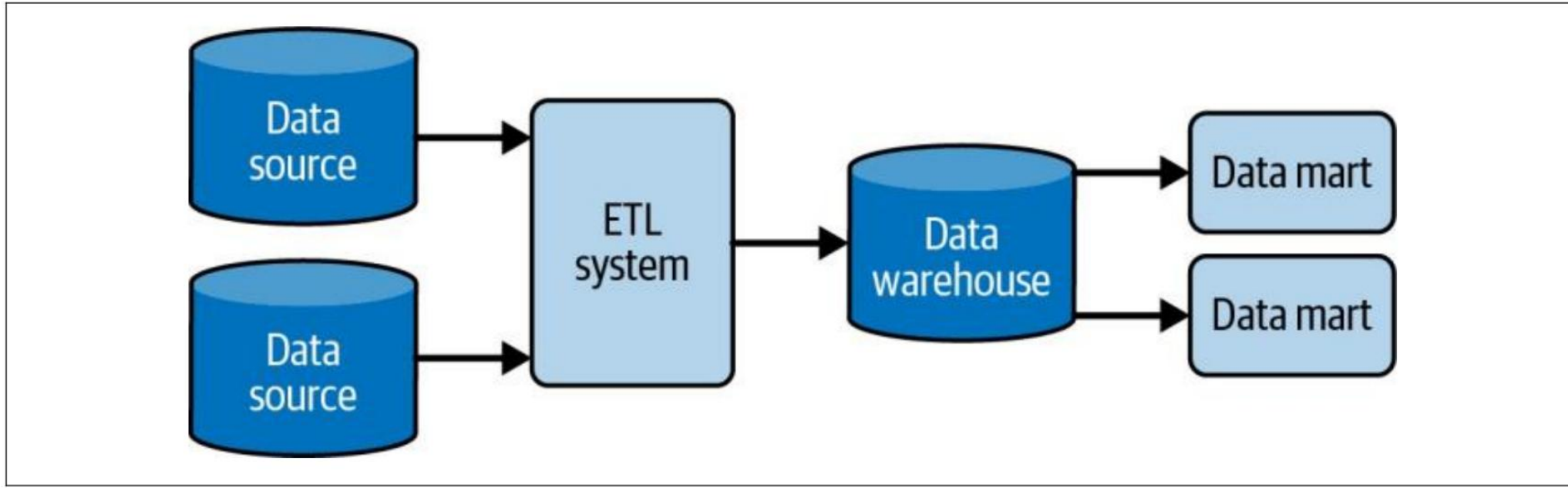


Figure 3-10. Basic data warehouse with ETL

Data Mart: Refined subset of data ware house.

Here further denormalized and transformed data specific to a subject area is present.

A data mart makes data more easily accessible to analysts and report developers.

Data marts provide an additional stage of transformation beyond that provided by the initial ETL or ELT pipelines.

Examples and Types of Data Architecture

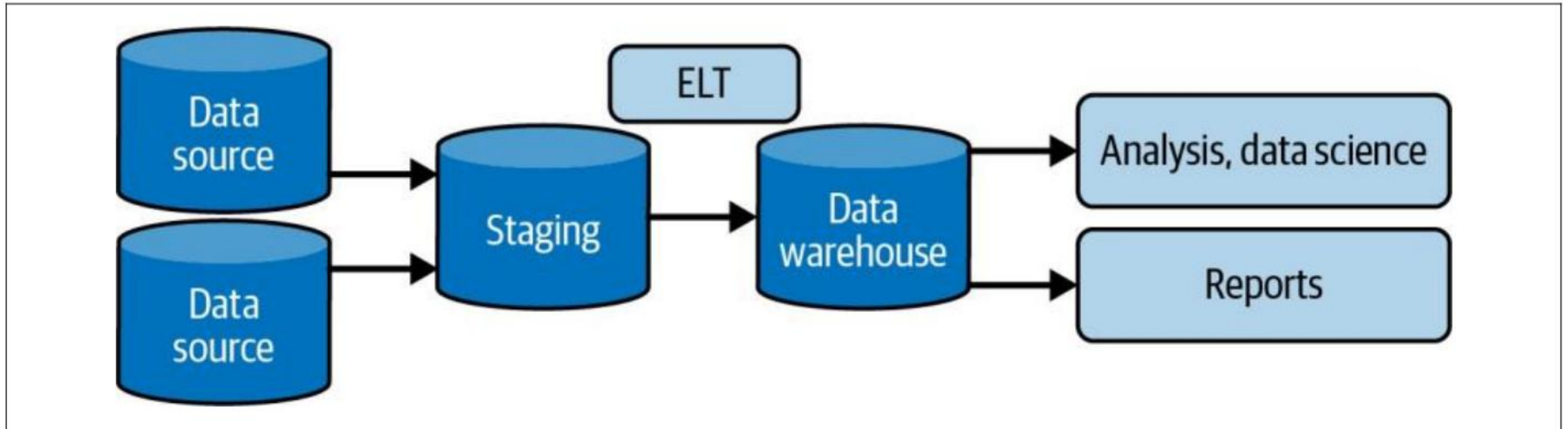


Figure 3-11. ELT—extract, load, and transform

Staging area: Contains raw data.

Examples and Types of Data Architecture

Evolution of Data warehouse



Traditional (On-Premises)

Used mainly by large enterprises.

Very expensive (millions of dollars).

Required large internal teams (DBAs, ETL developers).

Infrastructure was managed internally.



Modern (Cloud-Based)

Pay-as-you-go model.

Scalable and more affordable.

Infrastructure managed by cloud providers.

Smaller teams can manage complex data systems.

Examples and Types of Data Architecture

Feature	Data Warehouse	Data Lake	Data Lakehouse
Structured data	✓	✓	✓
Unstructured data	✗	✓	✓
ACID	✓	✗ (traditional)	✓
Schema-on-write	✓	✗	Hybrid
Schema-on-read	✗	✓	Hybrid
Storage cost	Higher	Low	Low
Centralized	Yes	Usually	Often

Examples and Types of Data Architecture

Lambda Architecture

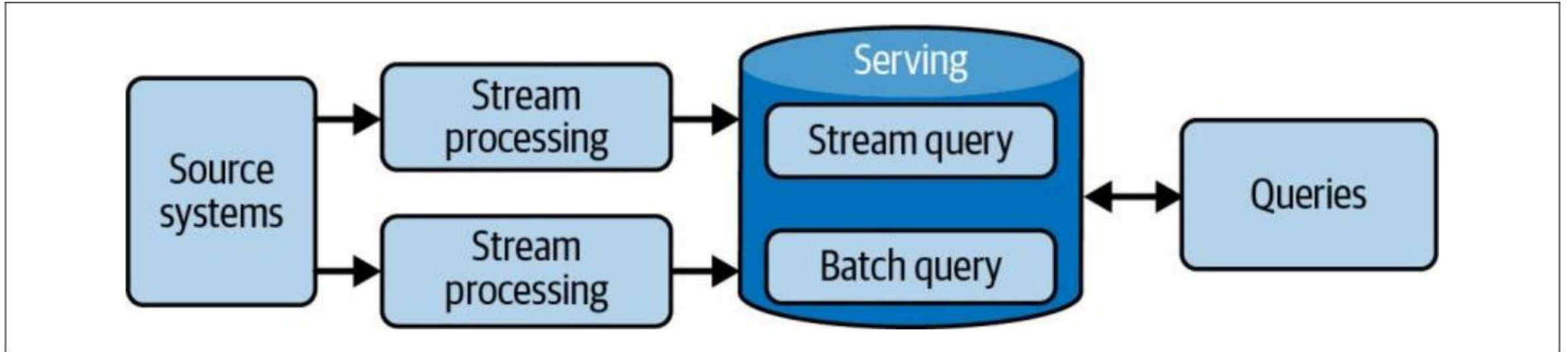


Figure 3-14. Lambda architecture

Examples and Types of Data Architecture

Lambda Architecture

Independent Layers: Lambda architecture separates processing into three independent systems:

- **Batch Layer** – handles large-scale, historical data processing.
- **Streaming Layer** – handles real-time, low-latency data.
- **Serving Layer** – provides combined query results from batch + streaming layers.

Data Source: Ideally immutable and append-only; data is sent to both batch and stream layers.

Main Idea: Combines speed (real-time) and accuracy (batch) in a single architecture.

Managing multiple systems with different codebases is as difficult as it sounds, creating error-prone systems with code and data that are extremely difficult to reconcile.

Examples and Types of Data Architecture

Kappa Architecture

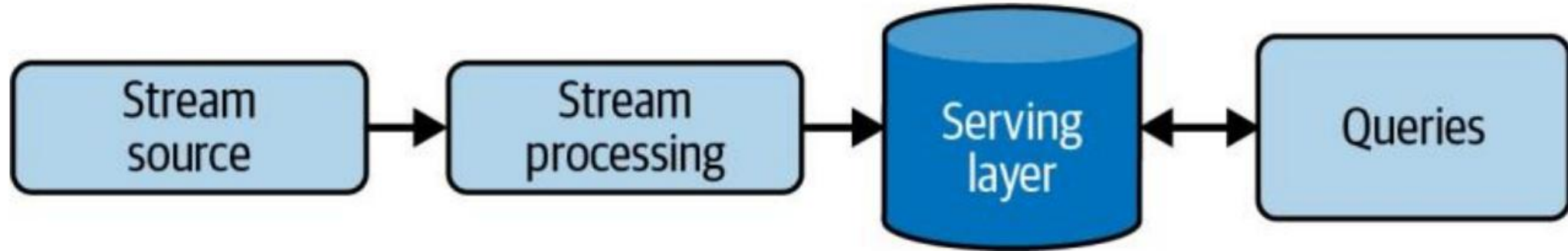


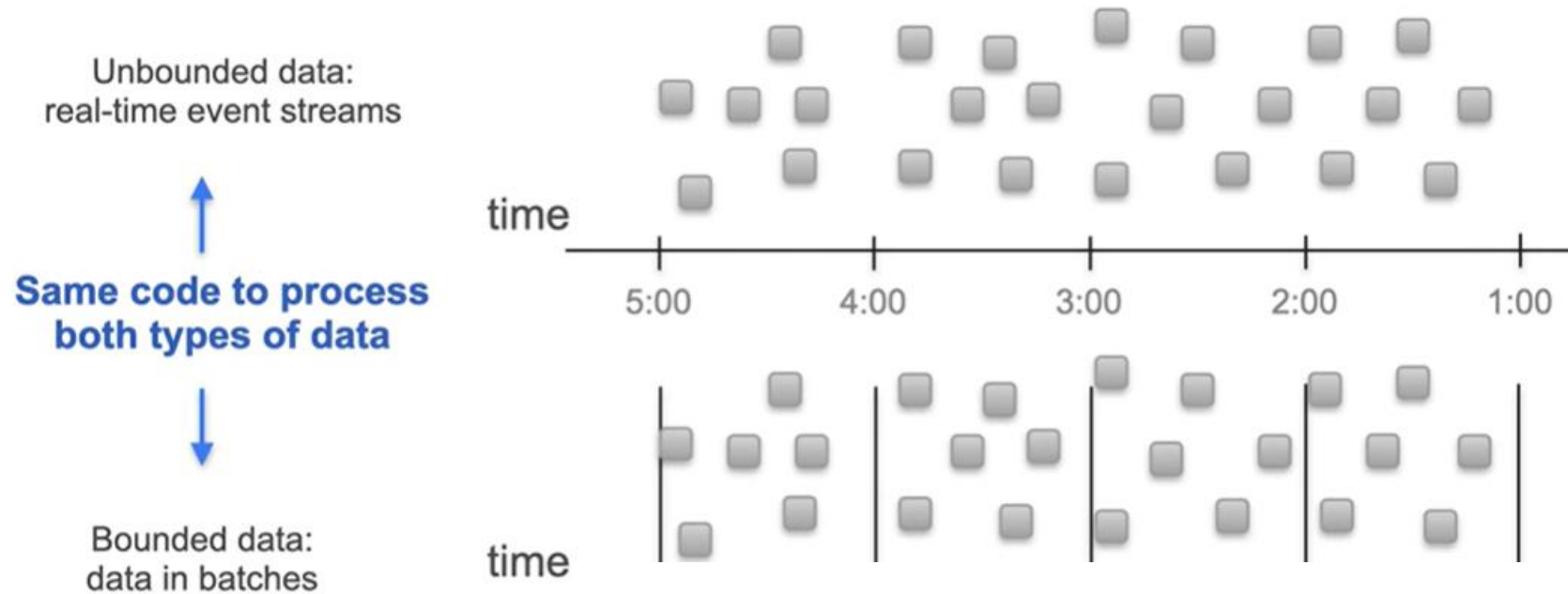
Figure 3-15. Kappa architecture

Real-time and batch processing can be applied seamlessly to the same data by reading the live event stream directly and replaying large chunks of data for batch processing.

Examples and Types of Data Architecture

The Dataflow Model and Unified Batch and Streaming

Unified Batch & Streaming



Examples and Types of Data Architecture

The Dataflow Model and Unified Batch and Streaming

Data as Events: Treats all data as a stream of events, whether real-time or batch

Bounded vs. Unbounded:

Unbounded streams → Real-time data with no fixed end.

Bounded streams → Batch data; boundaries create natural windows.

Windows for Aggregation:

Real-time aggregations are performed over windows.

Window types include:

Tumbling windows – fixed, non-overlapping intervals.

Sliding windows – overlapping intervals.

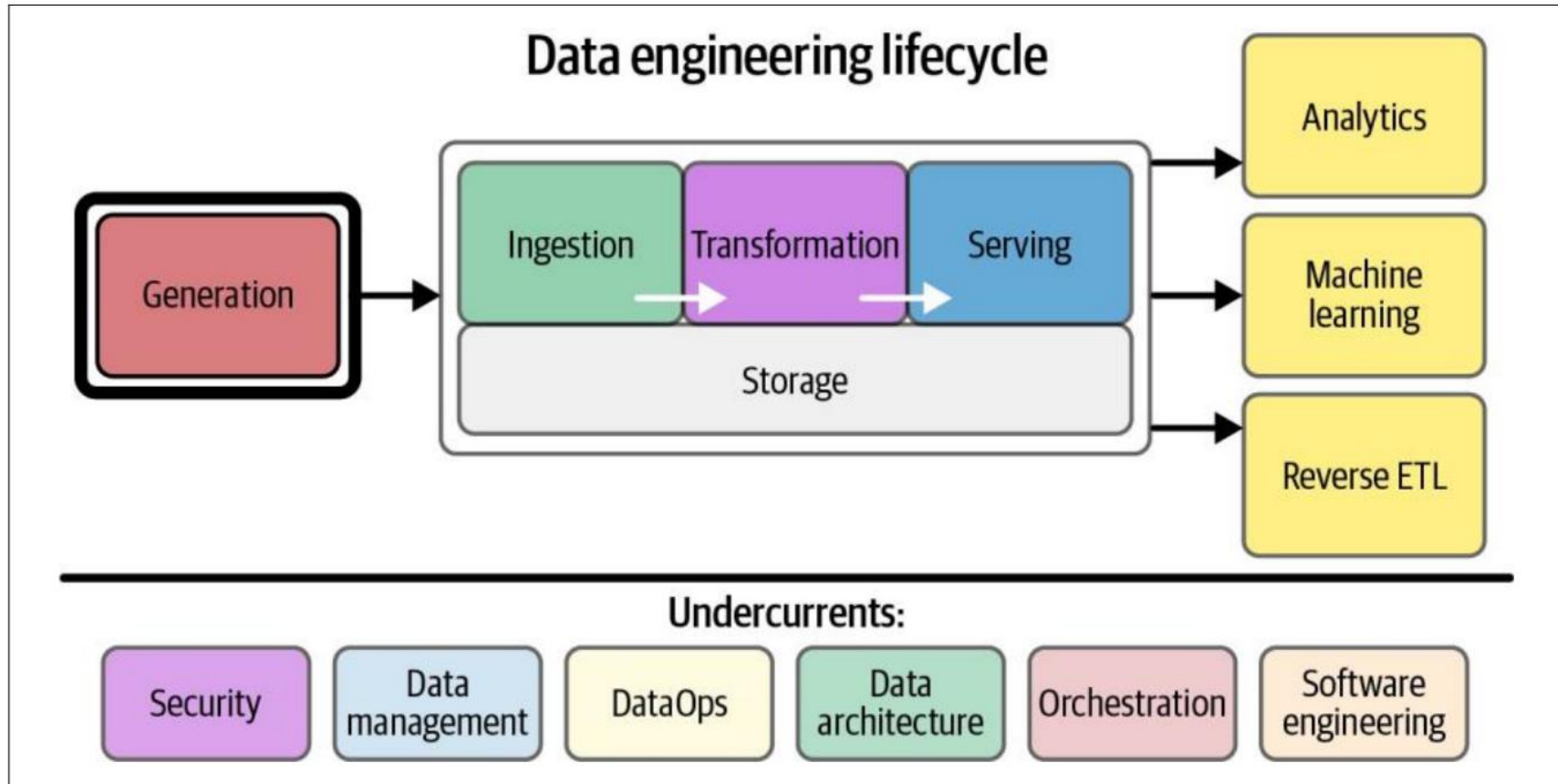
Unified Processing:

Both batch and real-time processing happen in the same system.

The same code can handle batch or streaming data.

Philosophy: “Batch is just a special case of streaming.”

Generation in Source Systems



Sources of Data

What is Data?

Data is a collection of raw facts and figures.
It is unorganized and has no meaning by itself.
It becomes meaningful only when given context.

Sources of Data

How is Data created?

Analog Data

Created in the physical world.

Examples:

Speaking in a conversation

Sign language

Writing on paper

Playing a musical instrument

Often temporary (transient).

Example: A verbal conversation that is not recorded and later forgotten.

Digital Data

a) Converting Analog to Digital

Real-world data is converted into digital form.

Example: A mobile app converting speech into text messages.

b) Born-Digital Data

Created directly by digital systems.

Examples:

Online credit card transactions, Ecommerce orders, Database records.

Source systems: Main Ideas

Files and Unstructured Data

What is a File?

A file is a sequence of bytes stored on a disk.

Applications use files to store data.

Files can contain:

- Settings or parameters
- Logs and events
- Images
- Audio

Files are a common way to share data between systems.

Files can contain different levels of structure:

Structured data

Excel

CSV

Semistructured data

JSON

XML

Sometimes CSV

Unstructured data

TXT

Sometimes CSV Others (e.g. Parquet, ORC)

Source systems: Main Ideas

Application Programming Interface APIs

What is an API?

A standard way for systems to exchange data.

Allows programs to communicate directly. APIs make data ingestion easier.

Data can be accessed programmatically.

In Practice

APIs can still be complex.

Engineers often need to:

Handle authentication

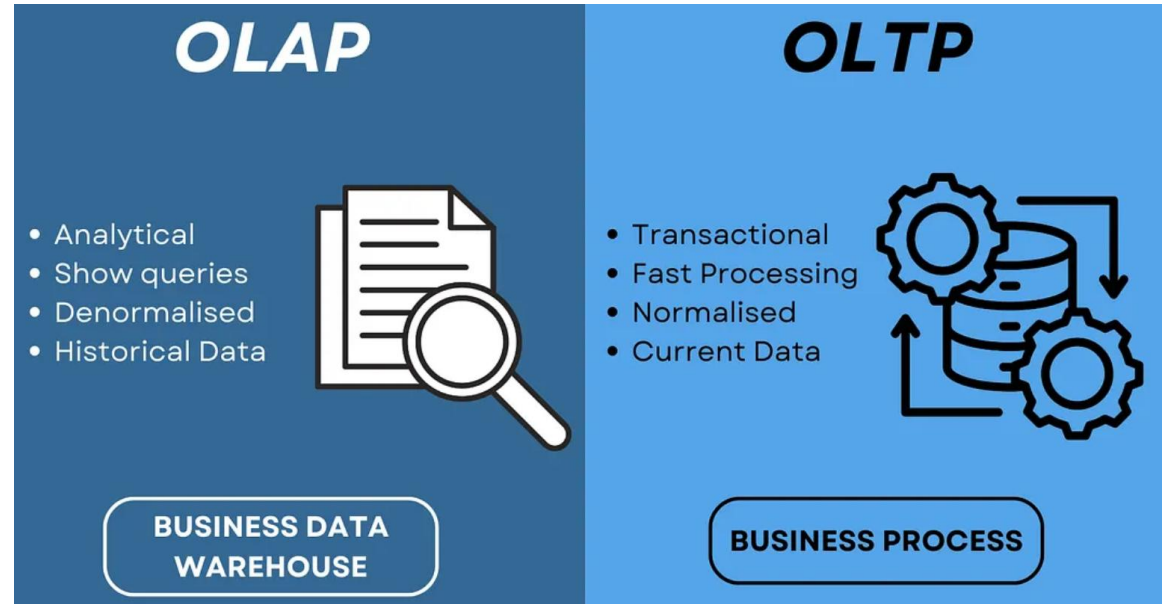
Manage rate limits

Source systems: Main Ideas

Data warehouse

Characteristics

- Separates OLAP from OLTP
- OLTP = transactional systems (production databases).
- OLAP = analytics systems
- Separating them:
 - Protects production systems.
 - Improves analytics performance.



Source systems: Main Ideas

Change Data Capture

CDC is a method for tracking changes in a database.

It captures every:

Insert (new record added)

Update (existing record modified)

Delete (record removed)

Instead of copying the whole database, it captures only the changes.

Source systems: Main Ideas

Logs

A log records events that happen in a system.

Examples:

- Operating system startup or crashes
- Application errors

Monitoring systems, Automation, Debugging



Source systems: Main Ideas

Logs

What information does a log contain?

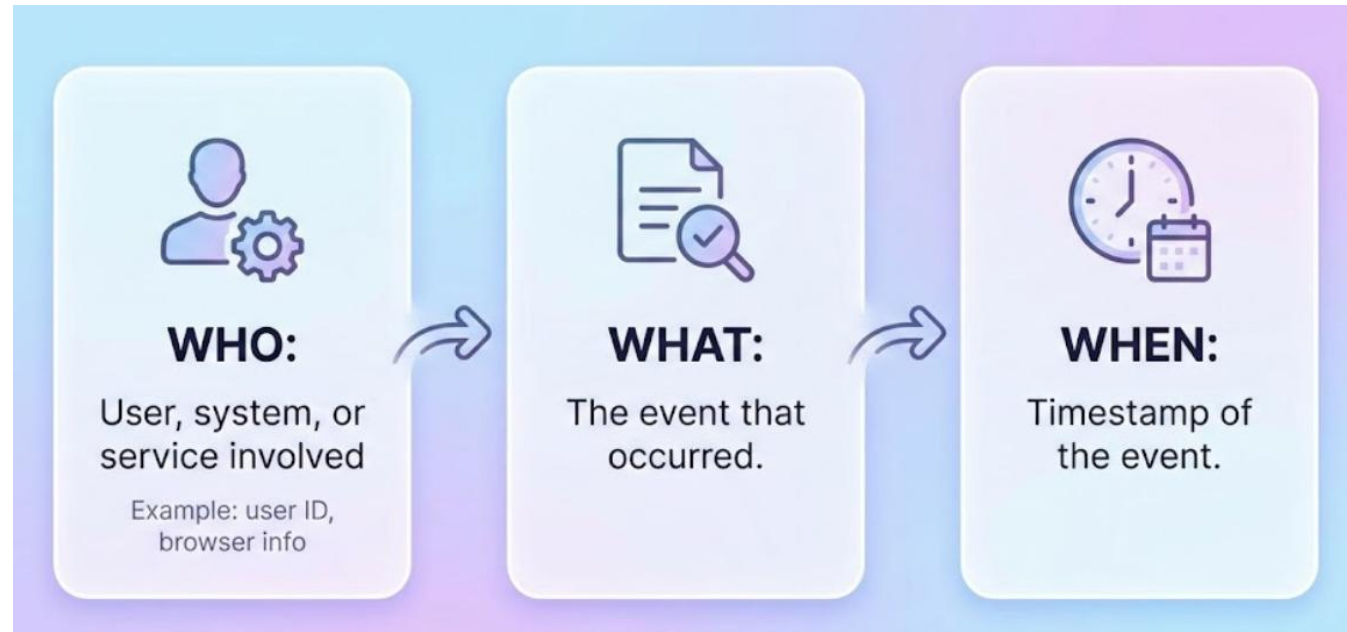
Who:

User, system, or service involved

Example: user ID, browser info

What: The event that occurred

When: Timestamp of the event



Source systems: Main Ideas

Logs: Types of Log Encoding

Binary Logs

Stored in compact format

Efficient and fast

Used in databases

Hard for humans to read

Semistructured Logs

Usually in JSON format

Machine-readable and portable

Easier to parse than plain text

Less efficient than binary

Plain-Text (Unstructured) Logs

Raw console output

No standard format

Harder to extract information

Often used for debugging

Log Resolution:

High resolution → very detailed (e.g., full database changes)

Low resolution → only major events recorded

Source systems: Main Ideas

Database Logs

Database logs record changes made to the database.

A common type is the Write-Ahead Log (WAL).

WAL files are usually:

- Stored in binary format
- Saved in a database-specific format

Write-Ahead Logging

When a write or update request is made:

- The database first writes the change to the log
- Then it updates the actual table
- Then it sends acknowledgment to the user

This ensures the change is safely recorded before being applied.

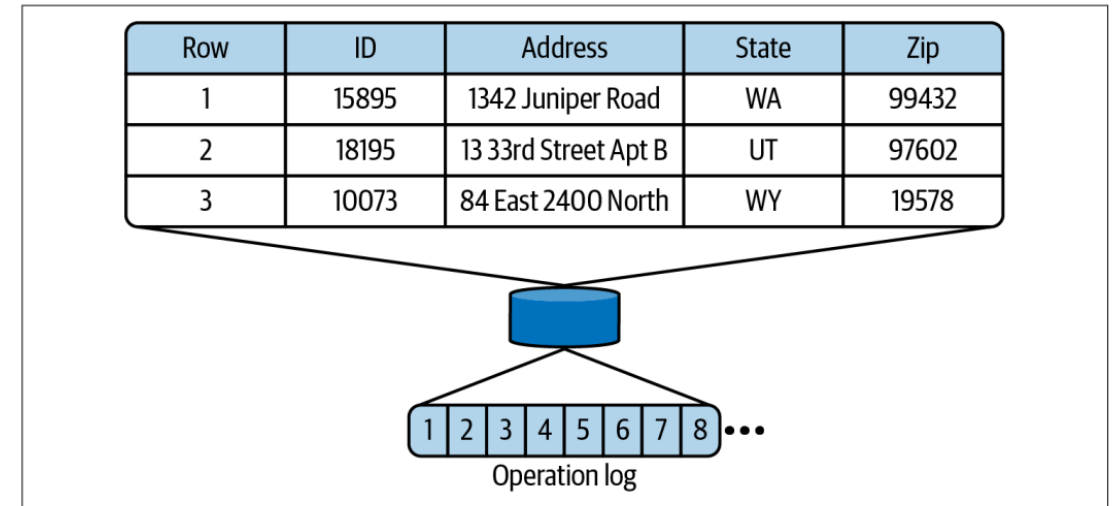


Figure 5-3. Database logs record operations on a table

Source systems: Main Ideas

CRUD (Create, Read, Update, Delete)

It represents the four basic operations for storing and managing data.

Data Extraction from CRUD Systems

Snapshot-based extraction

- Pulls the current state of the database
- No history of changes

Event extraction using CDC (Change Data Capture)

Reads database logs

Captures every insert, update, delete

Provides full history



Source systems: Main Ideas

Insert Only Pattern

Instead of updating records, new records are inserted.

Each record has a timestamp.

Old records are never modified — history is preserved.

Table 5-1. An insert-only pattern produces multiple versions of a record

Record ID	Value	Timestamp
1	40	2021-09-19T00:10:23+00:00
1	51	2021-09-30T00:12:00+00:00

Source systems: Main Ideas

CRUD	Insert-Only
Updates existing records	Always inserts new records
Only current state stored	Full history stored
Smaller table size	Larger table size
Faster lookup	Slower lookup for latest record

Source systems: Main Ideas

Message

A message is raw data sent from one system to another.

- Discrete (one event at a time)
- Short-lived
- Sent from a publisher to a consumer
- Delivered through a message queue.

How it works:

- System A sends a message.
- Message enters a queue.
- System B consumes the message.
- After processing, the message is removed.

Real-time actions and communication between systems.

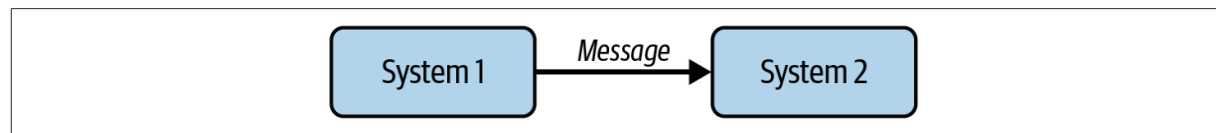


Figure 5-4. A message passed between two systems

Source systems: Main Ideas

Stream

A stream is an append-only log of events.

Events:

- Are stored in order (by timestamp or ID)
- Are kept for a long time (weeks/months)
- Are not removed after consumption

Properties:

- Persistent
- Ordered sequence of events
- Can “rewind” and replay past data
- Supports aggregation across many events

Streams are used when we want: Trends, History, Analytics across multiple events

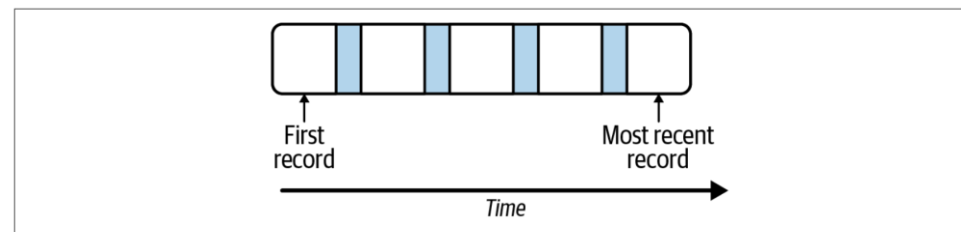


Figure 5-5. A stream, which is an ordered append-only log of records

Source systems: Main Ideas

Message Queue and Stream

Message Queue	Stream
One event at a time	Continuous sequence of events
Removed after processing	Stored for long retention
Focused on action	Focused on history & analytics
Short-term communication	Long-term event log

Source systems Main Ideas

Types of time data streaming

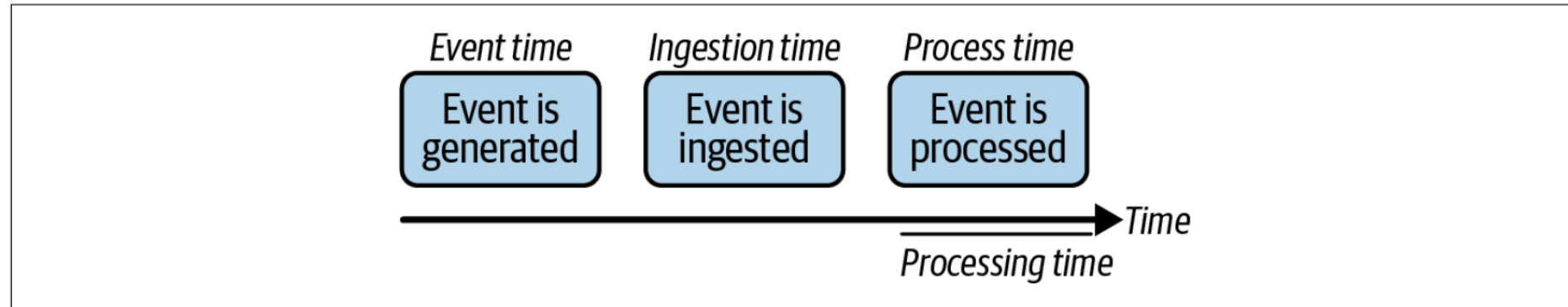


Figure 5-6. Event, ingestion, process, and processing time

Source systems: Main Ideas

Types of time data streaming

Event Time

- The time when the event actually happens in the source.
- Comes from the original event timestamp.

Example:

A sensor records temperature at 10:00:05 AM → that is event time.

Ingestion Time

The time when the event is received into your system.

For example, when it enters:
Message queue, Database, Object storage

Example:

Event happened at 10:00:05 AM
Arrived in Kafka at 10:00:08 AM
→ 10:00:08 AM is ingestion time.

Processing Time

The time when the data is actually processed or transformed.

Also refers to how long processing takes.

Example:

Ingested at 10:00:08 AM
Processed at 10:00:10 AM
Processing duration = 2 seconds

Source systems Main Ideas

Types of time data streaming

Event Time

- The time when the event actually happens in the source.
- Comes from the original event timestamp.

Example:

A sensor records temperature at 10:00:05 AM → that is event time.

Events may arrive late.
Networks cause delays.

Ingestion Time

The time when the event is received into your system.

For example, when it enters:
Message queue, Database, Object storage

Example:

Event happened at 10:00:05 AM
Arrived in Kafka at 10:00:08 AM
→ 10:00:08 AM is ingestion time.

Used to measure latency, detect bottlenecks, debug pipeline delays.

Processing Time

The time when the data is actually processed or transformed.

Also refers to how long processing takes.

Example:

Ingested at 10:00:08 AM
Processed at 10:00:10 AM
Processing duration = 2 seconds

Source systems: Practical Details

Databases

- Common source system database.
- There are as many types of databases as there are use cases for data.

Source systems: Practical Details

Databases : Relational Database Management Systems (RDBMS)

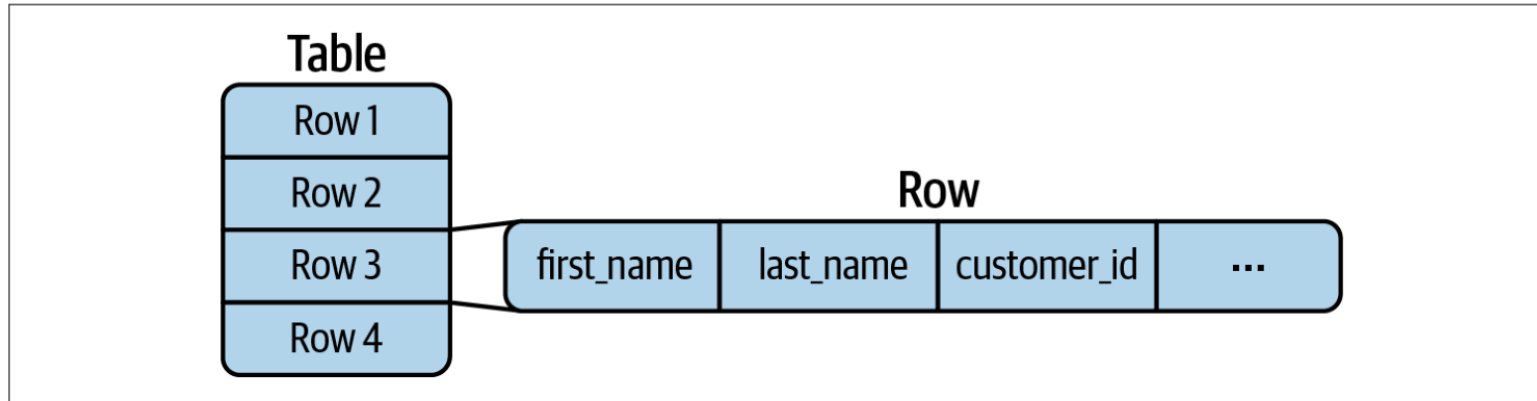


Figure 5-7. RDBMS stores and retrieves data at a row level

Source systems: Practical Details

Databases : Relational Database Management Systems (RDBMS)

- Data is stored in tables.
- Rows (also called relations)
- Columns (also called fields)
- Each row represents one record.
- Follows the same schema (same column structure).
- Each column has a fixed data type (e.g., string, integer, float).

Primary Key

- A unique identifier for each row.
- No two rows can have the same primary key.
- Used for indexing and fast lookup.

Foreign Key

- A field that links to the primary key of another table.
- Joins
- Complex schemas

Source systems: Practical Details

Databases : Relational Database Management Systems (RDBMS)

Normalization

- A database design strategy.
- Ensures data is not duplicated across tables.
- Stored efficiently.
- Example: Instead of repeating customer address in many tables, store it once and reference it via foreign key.

Most RDBMS follow ACID Principles

Atomicity – All-or-nothing transactions

Consistency – Database remains valid

Isolation – Transactions don't interfere

Durability – Committed data is not lost

Source systems: Practical Details

Databases : Non relational Databases (NoSQL)

NoSQL stands for “Not Only SQL.”

It refers to databases that:

- Do not follow the relational model
- Do not rely on tables with strict schemas

Designed for:

- High scalability
- Flexible data
- Large-scale distributed systems

Advantages:

- Better horizontal scalability
- More flexible schemas
- Improved performance for specific use cases

Trade-offs:

- Often weaker consistency guarantees
- Limited or no joins
- No fixed schema
- May not be fully ACID-compliant

Source systems: Practical Details

Databases : Key Value Store

Key-Value Store

Example: Web app session storage

When a user logs into Daraz:

Key → user_12345 Value → {loggedIn,
"Product A"}

The app quickly retrieves session data using the user ID.

Used for:

- Login sessions
- Shopping carts
- Caching frequently accessed data

Key:

user_101

Value:

"John, Premium User, 3 orders"

Source systems: Practical Details

Databases : Document Store

Document Store : Document JSON like

Example: User profile in a social media app

The app quickly retrieves session data using the user ID.

Used for:

- User profiles
- Product catalogs
- Blog platforms
- Mobile app backends

```
{  
  "user_id": 123,  
  "name": "abc",  
  "email": "abc@email.com",  
  "posts": [  
    {"id": 1, "content": "Hello"},  
    {"id": 2, "content": "Vacation time"}  
  ]  
}
```

Source systems: Practical Details

Databases : Document Store

Document Store : Document JSON like

Example: User profile in a social media app

The app quickly retrieves session data using the user ID.

Used for:

- User profiles
- Product catalogs
- Blog platforms
- Mobile app backends

```
{
  "user_id": 123,
  "name": "abc",
  "email": "abc@email.com",
  "posts": [
    {"id": 1, "content": "Hello"},
    {"id": 2, "content": "Vacation time"}
  ]
}
```

Table 5-2. Comparison of RDBMS and document terminology

RDBMS	Document database
Table	Collection
Row	Document, items, entity

Source systems: Practical Details

Databases : Wide column database

Wide column database

Example: Ecommerce order tracking

Used for:

- Large ecommerce systems
- Ad tech tracking
- IoT data at scale

```
{  
  "user_id": 123,  
  "name": "abc",  
  "email": "abc@email.com",  
  "posts": [  
    {"id": 1, "content": "Hello"},  
    {"id": 2, "content": "Vacation time"}  
  ]  
}
```

Source systems: Practical Details

Databases : Graph database

Graph database

Example: Social Media connections

Nodes: Users

Edges: "connected_to"

Query example:

"How many 2nd-degree connections does John have?"

"Suggest new connections based on mutual connections." "Graph

Used for:

- Social networks
- Fraud detection (who is connected to suspicious accounts?)
- Recommendation engine

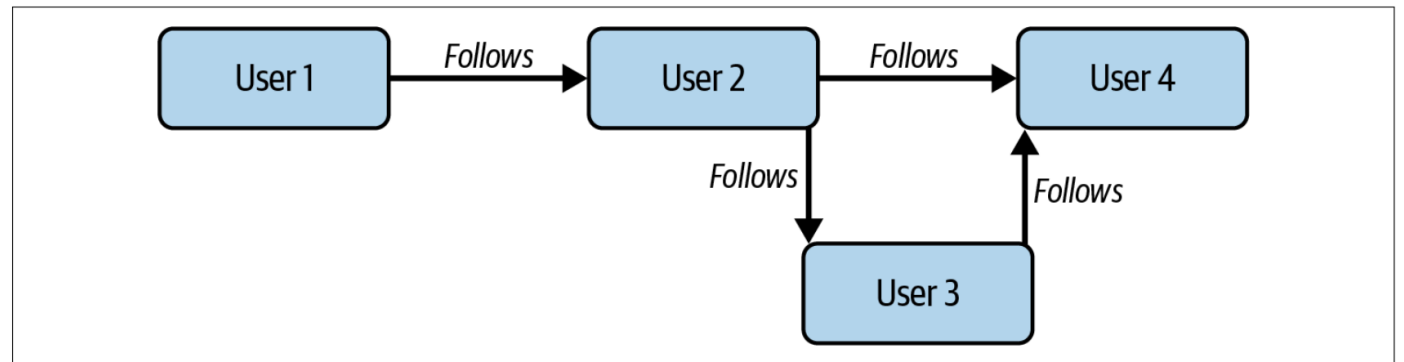


Figure 5-8. A social network graph

Source systems: Practical Details

Databases : Search database

Search database

Example: Product search

User types: "wireless noise cancelling
headphones"

Search database:

Matches exact words

Matches similar spellings

Ranks by relevance

Used for:

- Log analysis ("Find all errors in last 5 minutes")
- Monitoring systems
- Security event search

Source systems: Practical Details

Databases : Timeseries database

Time series database

Example: Weather monitoring system

Used for:

Stock prices

Application metrics

Server monitoring

Used for:

- timestamp: 10:01 → temperature: 72°F
- timestamp: 10:02 → temperature: 73°F
- timestamp: 10:03 → temperature: 72°F